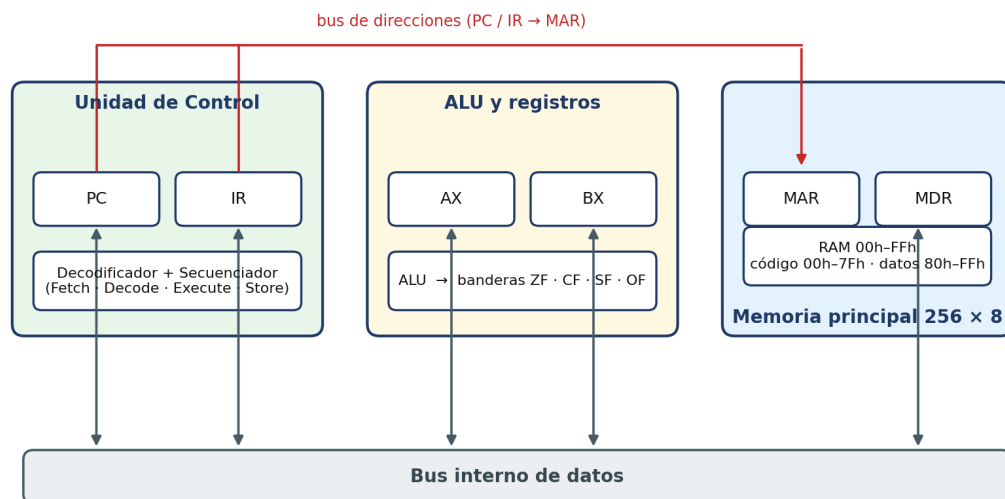


Simulador de CPU de 8 bits

Memoria principal de 256 bytes y ciclo de instrucción completo

Arquitectura Von Neumann · Google Sheets y Google Apps Script



Un solo espacio de memoria para instrucciones y datos (Von Neumann)

Figura 1. Arquitectura general del simulador.

SIS131 — Arquitectura de Computadoras

Josué Cartagena

Octubre de 2026

1. Introducción

Este informe describe el diseño, la implementación y la verificación de un simulador interactivo del ciclo de instrucción de una CPU de 8 bits con arquitectura Von Neumann. Está construido sobre Google Sheets (la interfaz) y Google Apps Script (la lógica), de modo que se ejecuta en el navegador y no requiere instalar nada.

El simulador reproduce las tres piezas de la lámina de clase: la Unidad de Control, la Unidad Aritmético-Lógica (ALU) y la Memoria Principal. Sobre ellas modela el ciclo Fetch → Decode → Execute → Store, un banco de 2 registros de propósito general (AX y BX), los registros internos PC, IR, MAR y MDR, cuatro banderas de estado (ZF, CF, SF y OF) y un repertorio de 22 instrucciones.

Además del simulador base, el proyecto incluye seis programas de demostración, un ensamblador de dos pasadas, un inspector de memoria con edición en vivo, una suite de 63 pruebas automatizadas y una hoja de equivalencias entre los nombres de la lámina y los del simulador.

1.1 Objetivos

- Mostrar paso a paso cómo una CPU busca, decodifica, ejecuta y almacena cada instrucción.
- Permitir escribir y cargar programas propios, no solo uno fijo.
- Hacer visibles los registros, las banderas y la memoria en cada micro-operación.
- Verificar el comportamiento con pruebas automáticas, no solo a ojo.

1.2 Tecnología

Google Sheets aporta la cuadrícula de memoria, el panel de registros y el log. Google Apps Script (JavaScript) aporta la lógica, repartida en 11 archivos con una responsabilidad cada uno (sección 11). El estado de la CPU se guarda en PropertiesService, de modo que persiste entre ejecuciones.

2. Arquitectura

La arquitectura Von Neumann usa una única memoria para las instrucciones y los datos, con un mismo espacio de direcciones. Es la diferencia clave frente a la arquitectura Harvard, donde el código y los datos viven en memorias y buses separados. Esta CPU es Von Neumann: por eso un programa puede, por ejemplo, leer como dato un byte que es parte de su propio código.

2.1 Memoria principal

La memoria tiene 256 posiciones de 8 bits (00h a FFh). Se muestra como una matriz de 16 × 16 y se divide lógicamente en dos segmentos, con colores distintos:

- Segmento de código, 00h–7Fh (azul): las instrucciones del programa.
- Segmento de datos, 80h–FFh (amarillo): variables de entrada y resultados.

Solo dos funciones tocan el arreglo interno de la memoria: readMem(dirección) y writeMem(dirección, valor). Todo el resto del simulador pasa por ellas, y cada acceso queda registrado en el log como una primitiva, por ejemplo Primitiva: Read(0x80) = 0x05.

2.2 Unidad de Control

Orquesta las cuatro fases del ciclo (fetchPhase, decodePhase, executePhase y storeBackPhase) y contiene la tabla de opcodes: qué mnemónico corresponde a cada código, cuántos bytes ocupa la instrucción y qué hace en la fase Execute.

2.3 ALU

Implementa ADD, SUB, INC, DEC, AND, OR, XOR, NOT y CMP. Cada operación recibe el objeto de banderas y lo actualiza. Las operaciones lógicas dejan siempre CF = 0 y OF = 0, porque esas banderas solo tienen sentido en aritmética.

2.4 Registros

Registro	Ancho	Función
PC (Program Counter)	8 bits	Dirección de la próxima instrucción.
IR (Instruction Register)	—	Opcode y operandos de la instrucción en curso.
MAR (Memory Address Register)	8 bits	Dirección de la celda que se lee o escribe.
MDR (Memory Data Register)	8 bits	Dato que entra o sale de la memoria.
AX (acumulador)	8 bits	Registro de propósito general; destino de la ALU.
BX	8 bits	Segundo registro de propósito general.

Solo hay dos registros de propósito general. No es un descuido: es una limitación real que explica por qué programas como la multiplicación (sección 7.2) deben usar celdas de memoria como almacenamiento auxiliar.

3. Banderas de estado

Bandera	Nombre	Se activa cuando...
ZF	Zero Flag	El resultado de la última operación fue exactamente 0.
CF	Carry Flag	Una suma pasó de 255, o una resta pidió prestado. Es una lectura sin signo.
SF	Sign Flag	El bit 7 del resultado es 1: se ve negativo en complemento a 2.
OF	Overflow Flag	El resultado, leído con signo, se salió del rango de 8 bits (−128 a 127).

3.1 Por qué OF es distinta de CF

Es el punto conceptual más fino del proyecto. Con un ejemplo: $100 + 50 = 150$.

- 150 es menor que 255, así que no hay acarreo sin signo: CF = 0.
- El bit 7 de 150 (10010110) es 1, así que SF = 1.
- En complemento a 2, 150 se lee como −106. Dos números positivos no pueden sumar uno negativo: eso es lo que mide OF, y por eso OF = 1 aunque CF = 0.

La regla para la suma es: OF = 1 si los dos operandos tienen el mismo signo y el resultado tiene signo distinto. Para la resta (y CMP): OF = 1 si los operandos tienen signos distintos y el resultado tiene un signo distinto al del primer operando. Este caso (100 + 50) está cubierto por una prueba automática.

4. Repertorio de instrucciones (ISA)

La codificación es de longitud variable: un byte de opcode seguido de 0, 1 o 2 bytes de operando. Los registros se codifican como AX = 0x00 y BX = 0x01.

Opcode	Mnemónico	Bytes	Descripción
0x00	HLT	1	Detiene el reloj.
0x01	MOV reg, imm	3	$\text{reg} \leftarrow \text{inmediato}$
0x02	MOV reg, reg	3	$\text{destino} \leftarrow \text{origen}$
0x03	LOAD reg, [dir]	3	$\text{reg} \leftarrow \text{RAM}[\text{dir}]$
0x04	STORE [dir], reg	3	$\text{RAM}[\text{dir}] \leftarrow \text{reg}$
0x05	ADD reg, imm	3	$\text{reg} \leftarrow \text{reg} + \text{inmediato}$; actualiza ZF, CF, SF, OF
0x06	ADD reg, reg	3	$\text{destino} \leftarrow \text{destino} + \text{origen}$; actualiza banderas
0x07	SUB reg, imm	3	$\text{reg} \leftarrow \text{reg} - \text{inmediato}$; actualiza banderas
0x08	SUB reg, reg	3	$\text{destino} \leftarrow \text{destino} - \text{origen}$; actualiza banderas
0x09	INC reg	2	$\text{reg} \leftarrow \text{reg} + 1$
0x0A	DEC reg	2	$\text{reg} \leftarrow \text{reg} - 1$
0x0B	CMP reg, imm	3	Calcula $\text{reg} - \text{inmediato}$; solo actualiza banderas
0x0C	CMP reg, reg	3	Calcula $\text{reg} - \text{reg}$; solo actualiza banderas
0x0D	AND reg, reg	3	$\text{destino} \leftarrow \text{destino AND origen}$
0x0E	OR reg, reg	3	$\text{destino} \leftarrow \text{destino OR origen}$
0x0F	XOR reg, reg	3	$\text{destino} \leftarrow \text{destino XOR origen}$
0x10	NOT reg	2	$\text{reg} \leftarrow \text{NOT reg}$
0x11	JMP dir	2	$\text{PC} \leftarrow \text{dir}$ (incondicional)
0x12	JZ dir	2	Si ZF = 1: $\text{PC} \leftarrow \text{dir}$
0x13	JNZ dir	2	Si ZF = 0: $\text{PC} \leftarrow \text{dir}$
0x14	JC dir	2	Si CF = 1: $\text{PC} \leftarrow \text{dir}$
0x15	JNC dir	2	Si CF = 0: $\text{PC} \leftarrow \text{dir}$

JC y JNC se agregaron al final de la tabla, sin reenumerar ni reutilizar opcodes existentes, para no invalidar ningún programa ya escrito. Hacen falta para escribir, por ejemplo, «Mayor de dos números» y la demostración de desbordamiento.

Cuidado con 0x06 y 0x08: ADD reg, reg es 0x06 y SUB reg, reg es 0x08. Confundirlos cambia el resultado de un programa sin producir ningún error (sección 9).

5. Ciclo de instrucción

Cada instrucción se ejecuta en cuatro fases, que se pueden recorrer una a una o de corrido:

- FETCH: $\text{MAR} \leftarrow \text{PC}$; $\text{MDR} \leftarrow \text{RAM}[\text{MAR}]$; $\text{IR} \leftarrow \text{MDR}$ (más los operandos según la longitud); $\text{PC} \leftarrow \text{PC} + \text{tamaño}$.

- En cada fase se resalta el registro o la celda activa y se agrega una línea al log de micro-operaciones con una explicación en lenguaje sencillo. El log distingue las fases por color.

Modo	Qué hace	Cuándo usarlo
STEP	Avanza una fase, con animación.	Estudiar el ciclo en detalle.
RUN	Ejecuta sin pintar y muestra el resultado de una vez.	Ver el resultado final.
Run Lento	Cada fase con 1 segundo de pausa.	Exposición en vivo.
PAUSE / RESET	Detiene el Run Lento / reinicia la CPU.	Control de la ejecución.

Toda la interfaz vive en la hoja CPU_SIMULATOR. A la izquierda están los registros y las banderas; en el centro, la matriz de memoria; abajo, el log y el diagrama de bloques.

[illegible]

26	LOG DE MICRO-OPERACIONES	FETCH	DECODE	EXECUTE	STORE	SISTEMA
27	— Programa "SUMA 1..N" cargado (N=0 en óx00, esperado en 0x01 = 15) —					
28	[Paso 0] FETCH: MAR=0x00, MDR=0x01 → IR=MOV AX, 0x00	1 4 0x16 Cont. Programa				
29	[Paso 0] DECODE: Opcode=0x01 → MOV AX, 0x00	5 HLT Decodificador				
30	[Paso 0] EXECUTE: MOV AX, 0x00 ZF=0 CF=0 SF=0 OF=0 // Carga el valor inmediato 0x00 en AX.	2 HLT R. Instrucciones				
31	[Paso 0] STORE: (esta instrucción no escribe en RAM)	7 Accumulador 0x0F (15d)				
32	[Paso 1] FETCH: MAR=0x03, MDR=0x03 → IR=LOAD BX, [0x00]	8 R. Entrada 0x00 (0d)				
33	[Paso 1] DECODE: Opcode=0x03 → LOAD BX, [0x00]	ZF CF SF 1 0 0				
34	Primaria: Read(0x00) = 0x05	6 Fetch Decode Execute Store				
35	[Paso 1] EXECUTE: LOAD BX, [0x00] ZF=0 CF=0 SF=0 OF=0 // Carga en BX el valor guardado en memoria[0x00]	Ciclos 31				
36	[Paso 1] STORE: (esta instrucción no escribe en RAM)	Memoria				
37	[Paso 2] FETCH: MAR=0x06, MDR=0x06 → IR=CMP BX, 0x00					
38	[Paso 2] DECODE: Opcode=0x06 → CMP BX, 0x00					
39	[Paso 2] EXECUTE: CMP BX, 0x00 ZF=0 CF=0 SF=0 OF=0 // Compara BX con 0x00 (actualiza banderas).					
40	[Paso 2] STORE: (esta instrucción no escribe en RAM)					
41	[Paso 3] FETCH: MAR=0x09, MDR=0x12 → IR=JZ 0x12					
42	[Paso 3] DECODE: Opcode=0x12 → JZ 0x12					

- Abrir la hoja y recargarla (F5): aparece el menú «CPU Simulador».
- Elegir un programa en Cargar programa. Los valores de entrada se escriben en N (suma 1..N), Valor A y Valor B.

- Ejecutar con Step, Run o Run Lento, y reiniciar con Reset.
- Para un programa propio, escribirlo en la hoja ENSAMBLADOR desde la celda A3 (una instrucción por fila) y usar Ensamblar y cargar.
- Inspeccionar celda de memoria muestra el valor de una dirección en hexadecimal, binario y decimal.
- Correr pruebas automatizadas ejecuta las 63 comprobaciones y muestra el resumen.

7. Programas de demostración

Los seis programas usan solo instrucciones de la tabla de la sección 4. Todos comparten una convención: el código siempre empieza en la dirección 00h y los datos van a partir de DATA_BASE (por defecto 0x80).

7.1 Suma de 1 a N

Programa base, con bucle y salto condicional: un contador regresivo (BX) y un acumulador (AX). Para N = 5 ejecuta 31 instrucciones: 2 de preparación, 5 vueltas de 5 instrucciones, 2 de la comprobación final, el STORE y el HLT.

```

0x00 MOV  AX, 0x00      ; AX = 0
0x03 LOAD BX, [0x80]    ; BX = N
0x06 CMP  BX, 0x00      ; L1: ¿BX = 0?
0x09 JZ   0x12          ; si es 0, salir
0x0B ADD  AX, BX        ; AX = AX + BX (byte 0x06)
0x0E DEC  BX           ; BX = BX - 1
0x10 JMP  0x06          ; volver a L1
0x12 STORE [0x81], AX   ; L2: resultado = AX
0x15 HLT

```

7.2 Multiplicación A × B

Multiplica por sumas sucesivas. Con solo dos registros no cabe a la vez el acumulador, el multiplicando y el contador, así que el programa guarda y recupera valores de memoria en cada vuelta.

7.3 C = A + B

El programa más simple: sin bucles, cuatro instrucciones (LOAD, LOAD, ADD, STORE).

7.4 Mayor de dos números

Compara con CMP AX, BX y salta con JNC: si CF = 0 no hubo préstamo, es decir A ≥ B.

4	REGISTROS				MEMORIA PRINCIPAL (00h - FFh)																
5	PC:	0x14			00h	0x03	0x01	0x80	0x03	0x04	0x81	0x0C	0x00	0x01	0x15	0x10	0x04	0x82	0x01	0x11	0x13
6	IR:	HLT			10h	0x04	0x82	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
7	MAR:	0x13			20h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
8	MDR:	0x00			30h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
9	AX:	0x0C (12d)			40h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
10	BX:	0x09 (9d)			50h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
11					60h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
12	FLAGS				70h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
13	ZF:	0			80h	0x00	0x09	0x0C	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
14	CF:	0			90h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
15	SF:	0			A0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
16	OF:	0			B0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
17	FASE ACTUAL:	HALT			C0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
18	CICLOS EJECUTADOS:	6			D0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
19					E0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
20	CONTOLES			ENTRADAS (prog E0h		0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
21	Velocidad RUN (ms):	400	Valor A:	12 F0h		0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00

Figura 4. Mayor de dos números con A = 12 y B = 9: el resultado 0x0C queda en 0x82.

7.5 Cuenta regresiva

Bucle DEC + JZ sin acumular nada: la forma más directa de ver el ciclo repetirse.

7.6 Fibonacci hasta desbordar 8 bits

Suma AX + BX en cada vuelta y corta con JC cuando el resultado pasa de 255. Con la secuencia real, $233 + 144 = 377$ desborda: queda 121 ($377 \bmod 256$), con CF = 1 y OF = 1 a la vez.

REGISTROS				MEMORIA PRINCIPAL (00h - FFh)																	
				0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F		
4	PC:	0x17		00h	0x01	0x00	0x01	0x01	0x01	0x04	0x80	0x00	0x06	0x00	0x01	0x14	0x13	0x03	0x01		
6	IR:	HLT		10h	0x80	0x11	0x06	0x04	0x81	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
7	MAR:	0x16		20h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
8	MDR:	0x00		30h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
9	AX:	0x79 (121d)		40h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
10	BX:	0x90 (144d)		50h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
11				60h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
12	FLAGS			70h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
13	ZF:	0		80h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
14	CF:	1		90h	0xE9	0x79	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
15	SF:	0		A0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
16	OF:	1		B0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
17	FASE ACTUAL:	HALT		C0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
18	CICLOS EJECUTADOS:	62		D0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
19				E0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
20	CONTROLES		ENTRADAS (prog E0h)	F0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		
21	Velocidad RUN (ms):	400	Valor A: 12 F0h		0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00		

Figura 5. Fibonacci: AX = 0x79 (121), CF = 1 y OF = 1 tras 62 instrucciones.

7.7 Dónde queda cada dato

Todas las direcciones de datos se calculan como DATA_BASE + un desplazamiento. Con DATA_BASE = 0x80:

Programa	+0	+1	+2	+3	+4
Suma 1..N	N	Resultado			
Multiplicación	A	B	Acumulador	Contador	Resultado
C = A + B	A	B	C		
Mayor de dos	A	B	Máximo		
Cuenta regresiva	N	Resultado			
Fibonacci	Temporal	Último valor			

7.8 Direcciones configurables (DATA_BASE)

Un programa de memoria tiene tres lugares donde aparece una dirección de datos: la instrucción que lee (LOAD), la que escribe (STORE) y la carga inicial de los valores. Si una de las tres no coincide con las otras, el programa corre sin error pero lee o escribe donde no debe. Para evitarlo, las tres salen de una sola definición en Program.gs:

```
var DATA_BASE = 0x80; // único lugar a cambiar
```

Para mover los datos, por ejemplo a 0x90, se cambia esa línea y se vuelve a cargar el programa. El código, los datos y el log se ajustan juntos. Además hay una validación: si la dirección cae dentro del segmento de código o se pasa de 0xFF, el programa no se carga y el log explica el motivo.

8. Ensamblador e inspector

8.1 Ensamblador de dos pasadas

Permite escribir un programa como texto, con mnemónicos, etiquetas y comentarios, en vez de bytes. La primera pasada calcula las direcciones y recoge las etiquetas; la segunda resuelve los operandos y emite los bytes. Admite números en hexadecimal (0x1A) y decimal, comentarios con punto y coma, etiquetas (L1:) y las directivas ORG y DB. Si hay un error, se indica la línea.

```
MOV    AX, 0x00
LOAD   BX, [0x80]
L1:    CMP    BX, 0x00
      JZ     L2
      ADD    AX, BX
      DEC    BX
      JMP    L1
L2:    STORE  [0x81], AX
      HLT
```

Se verificó reensamblando dos programas y comparando el resultado byte a byte con el original: coinciden exactamente. La ventaja práctica es que, al usar etiquetas, agregar o quitar una instrucción no obliga a recalcular los saltos a mano.

8.2 Edición en vivo e inspector

Escribir un valor (decimal o hexadecimal) sobre una celda de la matriz de memoria cambia esa dirección al instante, sin recargar el programa. Un valor inválido se rechaza y se restaura el anterior. El inspector muestra una dirección en hexadecimal, binario, decimal con y sin signo, y cómo se leería si fuera una instrucción.

La edición en vivo cambia la memoria, pero Cargar programa la vuelve a escribir entera: un cambio hecho en la cuadrícula se pierde si después se recarga el programa.

9. Verificación

9.1 Pruebas automatizadas

El archivo Tests.gs ejecuta 63 comprobaciones usando las funciones reales del simulador, no una copia aparte. Corre sin tocar la hoja ni el estado guardado, de modo que se puede ejecutar en cualquier momento.

- ALU: suma, resta, comparación y lógicas, incluidos los casos $100 + 50$ (CF = 0, OF = 1) y $0x80 - 1$.
- Ensamblador: reensambla dos programas y compara el bytecode; comprueba que un mnemónico o una etiqueta inválidos den error.
- Programas: los seis, ejecutados de principio a fin con datos en 0x80, 0x90 y 0xC0, para comprobar que DATA_BASE mantiene todo coherente.
- Validación de direcciones: datos dentro del código o pasados de 0xFF se rechazan.

Resultado: 63/63 comprobaciones correctas, tanto en un entorno de pruebas independiente como ejecutadas dentro del propio proyecto de Apps Script.

9.2 Comprobación en la hoja real

Además de las pruebas automáticas, se cargaron y ejecutaron programas directamente en la hoja de Google Sheets:

Programa	Resultado observado	Detalle
Suma de 1 a N (N = 5)	0x0F (15) en 0x81	31 instrucciones; AX = 0x0F
Mayor de dos (12, 9)	0x0C (12) en 0x82	Usa CMP y JNC
Fibonacci	0x79 (121) en 0x81	62 instrucciones; CF = 1 y OF = 1

9.3 Un error real: ADD y SUB

Durante una revisión del programa Suma de 1 a N, el resultado en 0x81 era 0xF1 (241) en lugar de 0x0F (15). La causa fue un solo byte: en la dirección 0x0B el código tenía 0x08 (SUB AX, BX) donde debía tener 0x06 (ADD AX, BX). El comentario del código decía ADD, pero el byte restaba.

REGISTROS				MEMORIA PRINCIPAL (00h - FFh)															
				0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
4	PC:	0x16	00h	0x01	0x00	0x00	0x03	0x01	0x00	0x00	0x01	0x00	0x12	0x12	0x08	0x00	0x01	0x0A	0x01
5	IR:	HLT	10h	0x11	0x06	0x04	0x81	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
6	MAR:	0x15	20h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
7	MDR:	0x00	30h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
8	AX:	0xF1 (241d)	40h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
9	BX:	0x00 (0d)	50h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
10			60h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
11			70h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
12	FLAGS		80h	0x05	0xF1	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
13	ZF:	1	90h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
14	CF:	0	A0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
15	SF:	0	B0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
16	OF:		C0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
17	FASE ACTUAL:	HALT	D0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
18	CICLOS EJECUTADOS:	31	E0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
19			F0h	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
20	CONTROLES			0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00
21	Velocidad RUN (ms):	400		0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00	0x00

Figura 6. Memoria con 0x08 en la dirección 0x0B: el programa resta y AX termina en 0xF1.

La cuenta lo confirma: $0 - 5 - 4 - 3 - 2 - 1 = -15$, que en 8 bits es $241 = 0xF1$. Se corrigió el byte a 0x06 y la prueba automática de la Suma exige que el resultado sea 15, de modo que el error no pueda volver sin que las pruebas lo detecten.

La lección es que un comentario no ejecuta nada: lo que vale es el byte. Por eso conviene comprobar siempre el resultado final contra el valor esperado.

10. Estructura del código

Archivo	Tarjeta	Contenido
Code.gs	#2 Memoria y Registros	Estado de la CPU, lectura y escritura de memoria, pintado de la hoja, log.
ALU.gs	#3 ALU / #7 OF	Operaciones aritméticas y lógicas y sus banderas.
Control.gs	#4 Ciclo / #8 JC-JNC	Tabla de opcodes y las cuatro fases del ciclo.
Program.gs	#5 Programa de prueba / #13	Suma de 1 a N y DATA_BASE.
Program2.gs	#9 Programas	Los otros cinco programas.
Setup.gs	#6 Botones y visual	Construcción de la hoja, menú y reinicio.
Datapath.gs	#6 Botones y visual	Diagrama de bloques y Run Lento.

Archivo	Tarjeta	Contenido
Assembler.gs	#10 Ensamblador	Ensamblador de dos pasadas.
Inspector.gs	#11 Inspector	Edición en vivo e inspector de memoria.
Tests.gs	#12 Pruebas	Las 63 comprobaciones automáticas.
Teoria.gs	#14 Hoja Teoría	Equivalencias entre la lámina y el simulador.

Cada función lleva sobre ella un comentario con la tarjeta a la que pertenece, y cada archivo lista las tarjetas en su encabezado.

11. Gestión del proyecto

El trabajo se organizó en un tablero Kanban de GitHub Projects (Arquitectura-Project). Cada tarjeta es una issue y avanza por las columnas Backlog, In progress y Done.

Tarjeta	Nombre	Qué entrega
#1	Documentación y ensayo final	Este informe y el README.
#2	Memoria y Registros	Memoria de 256 bytes, registros y su cuadrícula.
#3	ALU (operaciones)	ADD, SUB, INC, DEC, AND, OR, XOR, NOT, CMP y banderas.
#4	Ciclo de instrucción	Fetch, Decode, Execute y Store; STEP y RUN.
#5	Programa de prueba (Suma 1..5)	Programa con bucle y salto condicional.
#6	Botones y visual	Menú, colores, log y diagrama de bloques.
#7	Bandera OF	Desbordamiento con signo y su explicación frente a CF.
#8	Salto JC / JNC	Dos opcodes nuevos al final de la tabla.
#9	Programas de demostración	Multiplicación, C = A + B, Mayor de dos, Cuenta regresiva y Fibonacci.
#10	Ensamblador de dos pasadas	Programas en texto con etiquetas, ORG y DB.
#11	Inspector y edición en vivo	Cambiar la memoria desde la cuadrícula e inspeccionar celdas.
#12	Pruebas automatizadas	63 comprobaciones sobre ALU, ensamblador y programas.
#13	Direcciones configurables	DATA_BASE y validación de direcciones.
#14	Hoja Teoría	Equivalencias de nombres y documentación actualizada.

12. Limitaciones y trabajo futuro

- La tabla de opcodes no tiene una codificación sistemática (nibble alto = operación, nibble bajo = modo). Rediseñarla invalidaría todos los programas existentes, así que JC y JNC se agregaron al final.
- El diagrama de bloques ilumina las cajas de la fase activa, pero no anima el dato viajando por los buses. En su lugar, el log muestra las primitivas Read y Write de memoria.
- Solo hay dos registros de propósito general, lo que obliga a usar memoria auxiliar en programas largos. Es una limitación del modelo, no un error.

- Una futura mejora natural es agregar constantes con nombre al ensamblador (por ejemplo ENTRADA EQU 0x80) para que las direcciones de datos también se cambien en un solo lugar dentro de los programas escritos en texto.

13. Conclusiones

El simulador cumple su propósito: hace visible, paso a paso, cómo una CPU de arquitectura Von Neumann busca, decodifica, ejecuta y almacena cada instrucción, con registros, banderas y memoria a la vista. Las partes añadidas (bandera OF, programas, ensamblador, inspector, direcciones configurables y pruebas) lo convierten de una demostración fija en una herramienta con la que se pueden escribir y verificar programas propios.

El caso del byte 0x08 en lugar de 0x06 resume la idea central del proyecto: en una CPU, lo que cuenta es el código de máquina, no lo que dice el comentario. Verificar el resultado de cada programa contra un valor esperado, y automatizar esa verificación, es lo que evita que un error silencioso llegue a la defensa.

Bibliografía

- Stallings, W. (2019). Computer Organization and Architecture: Designing for Performance (11.^a ed.). Pearson.
- Patterson, D. A., y Hennessy, J. L. (2020). Computer Organization and Design: The Hardware/Software Interface (6.^a ed.). Morgan Kaufmann.
- Mano, M. M., y Ciletti, M. D. (2013). Digital Design (5.^a ed.). Pearson.
- Material de clase de SIS131 — Arquitectura de Computadoras: ciclo de instrucción, Unidad de Control y arquitectura de Von Neumann.
- Google. Documentación de Google Apps Script: SpreadsheetApp, PropertiesService y triggers simples. developers.google.com/apps-script